

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

VISHWAJIT ANAND (1WA23CS046)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Bio Inspired Systems (23CS5BSBIS)” carried out by **Vishwajit Anand(1WA23CS046)**, who is Bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above-mentioned subject and the work prescribed for the said degree.

Sandhya A Kulkarni Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	
1	18/8/25	Genetic Algorithm	
2	25/8/25	Particle Swarm Optimization	
3	1/9/25	Ant Colony Optimization	
4	8/9/25	Cuckoo Search (CS)	
5	15/9/25	Grey Wolf Optimizer (GWO)	
6	29/9/25	Parallel Cellular Algorithms and Programs	
7	13/10/25	Gene Expression Algorithm	

Github Link:

<https://github.com/AnandVishwajit/bis>

Program 1

Genetic Algorithm for Optimization Problems

Algorithm:

```

- f(x) = x^2
169
576

729
289

- f(x) = x^2
84)
576
729
400

Application →
import random
def fitness(chroms: int) → int:
    return -(chroms**2) + 10*(chroms-100)

POP_SIZE = 50
GENS = 100
MUT_RATE = 0.05
CROSS_RATE = 0.7
CHROM_LEN = 10

to_int = lambda b: int(b, 2)
to_bin = lambda n: format(n, '0' + str(CHROM_LEN) + 'b')

def init_pop(size): return [''.join(random.
    choice('01') for _ in range(CHROM_LEN))
    for _ in range(size)]

def evaluate(pop): return [fitness(to_int(
    ind)) for ind in pop]

def select(pop, fits):
    return [max(random.sample(list(
        zip(pop, fits)), 3), key=lambda t:
        t[1])[0] for _ in pop]

def mutate(ind):
    return ''.join('1' if (y == '0' and
        random.random() < MUT_RATE) else
        '0' if (y == '1' and random.
        random() < MUT_RATE) else
        y for y in ind)

```

```
def crossover(p1, p2):  
    if random.random() < CROSS_RATE:  
        pt = random.randint(1, (len(p1) - 1))  
        return p1[:pt] + p2[pt:]  
        p2[:pt] + p1[pt:]  
    return p1, p2
```

```
def run_gen():  
    pop, best_fit, best_sol = init_pop(  
        POP_SIZE, float('-inf'), None
```

```
    for gen in range(1, GEN+1):  
        fits = evaluate(pop)  
        best_idx = min(range(len(fits)),  
                        key=fits.__getitem__)  
        if fits[best_idx] > best_fit:  
            best_fit, best_sol = fits[  
                best_idx], pop[best_idx]
```

```
    selected = select(pop, fits)  
    next_gen = []
```

```
    for i in range(0, POP_SIZE):  
        c1, c2 = crossover(selected[i],  
                           selected[i+1])
```

```
    next_gen += [mutate(c1),  
                 mutate(c2)]
```

```
    pop = next_gen  
    print(f"Gen {gen:03}")
```

```
    Best_solution = {best_fit}  
    (Trails = {to_int(best_sol)})
```

Code:

```
import random
import math
from dataclasses import dataclass
from typing import List, Tuple

# ----- Helpers -----
BIT_LEN = 5 # 5-bit chromosome
LOW, HIGH = 0, 31

def encode(x: int) -> str:
    return format(x, f'0{BIT_LEN}b')

def decode(bits: str) -> int:
    return int(bits, 2)

def fitness(x: int) -> int:
    return x * x

def single_point_crossover(p1: str, p2: str, point: int) -> Tuple[str, str]:
    return p1[:point] + p2[point:], p2[:point] + p1[point:]

def mutate(bits: str, rate: float) -> str:
    out = []
    for ch in bits:
        if random.random() < rate:
```

```

        out.append('1' if ch == '0' else '0')
    else:
        out.append(ch)
    return "".join(out)

```

----- Roulette Selection -----

```

def roulette_select(pop: List[str]) -> List[str]:
    xs = [decode(b) for b in pop]
    fs = [fitness(x) for x in xs]
    total = sum(fs)
    if total == 0:
        return random.choices(pop, k=len(pop))
    probs = [f / total for f in fs]
    # cumulative distribution
    cum = []
    acc = 0.0
    for p in probs:
        acc += p
        cum.append(acc)
    sel = []
    for _ in range(len(pop)):
        r = random.random()
        for i, c in enumerate(cum):
            if r <= c:
                sel.append(pop[i])
                break
    return sel

```

----- GA Config -----

```

@dataclass
class GAConfig:
    population_size: int = 12
    bit_len: int = BIT_LEN
    crossover_rate: float = 0.9
    mutation_rate: float = 1.0 / BIT_LEN
    generations: int = 20

```

----- Main Evolution -----

```

def evolve(config: GAConfig, seed_pop: List[str] = None) -> Tuple[List[str], List[Tuple[int,int]]]:
    if seed_pop is None:
        pop = [encode(random.randint(LOW, HIGH)) for _ in range(config.population_size)]
    else:
        pop = seed_pop[:]
        while len(pop) < config.population_size:
            pop.append(encode(random.randint(LOW, HIGH)))
        pop = pop[:config.population_size]

    history = []
    for g in range(config.generations):

```

```

xs = [decode(b) for b in pop]
fs = [fitness(x) for x in xs]
best = max(zip(xs, fs), key=lambda t: t[1])
history.append(best)

# Selection
mating = roulette_select(pop)

# Crossover
next_pop = []
for i in range(0, config.population_size, 2):
    p1, p2 = mating[i], mating[i+1]
    if random.random() < config.crossover_rate:
        point = random.randint(1, config.bit_len - 1)
        c1, c2 = single_point_crossover(p1, p2, point)
        next_pop.extend([c1, c2])
    else:
        next_pop.extend([p1, p2])

# Mutation
next_pop = [mutate(b, config.mutation_rate) for b in next_pop]
pop = next_pop

# final best
xs = [decode(b) for b in pop]
fs = [fitness(x) for x in xs]
best = max(zip(xs, fs), key=lambda t: t[1])
history.append(best)
return pop, history

# ----- Run -----
if __name__ == "__main__":
    random.seed(42)
    cfg = GAConfig(population_size=12, generations=30)

    # initial population (optional, from board)
    init_bits = ["01100", "11001", "00101", "10011"]

    final_pop, hist = evolve(cfg, seed_pop=init_bits)

    print("Final Population:")
    for b in final_pop:
        x = decode(b)
        print(f'{b} -> x={x}, fitness={fitness(x)}')

    print("\nBest per generation:")
    for gen, (x, f) in enumerate(hist):
        print(f'Gen {gen}: x={x}, fitness={f}')

```


Program 2

Particle Swarm Optimization for Function Optimization

Algorithm:

Particle Swarm Optimization

It is like a flock of birds moving towards the food, where every bird (particle) has its own best position and in general move towards the globally available best position.

$$\text{Fitness function} = \sum_{i=1}^n (x_i - p_i)^2$$

```
import random
```

```
class Particle:
```

```
def __init__(self, dim, bounds)
```

```
    self.position = [random.  
        randint(bounds[i][0],  
        bounds[i][1]) for i  
        in range(dim)]
```

```
    self.velocity = [0] * dim
```

```
    self.best_position = list(  
        self.position)
```

```
    self.best_score = float('inf')
```

```
def update_velocity(self,
                    global_best, w=0.5, c1=1.5,
                    c2=1.5)
```

```
    for i in range(len(self.velocity)):
        r1, r2 = random.random(),
                  random.random()
        cognitive = c1 * r1 * (
            self.best_position[i]
            - self.position[i])
```

```
        social = c2 * r2 * (
            self.best_position[i]
            - self.position[i])
```

```
        self.velocity[i] = ind(
            w * self.velocity[i] +
            cognitive + social)
```

```
def update_position(self, bounds):
    for i in range(len(self.position)):
        self.position[i] += self.velocity[i]
        self.position = max(bounds[0], min(bounds[1], self.position[i]))
```

```
preferences = [3, 4, 2]
total_shifts = sum(preferences)
hours = [0, total_shifts] * len(preferences)
```


def fitness (Schedule):
 if sum (Schedule) != total_shifts
~~return float ('inf')~~
 return float ('inf')
 return sum (1 Schedule[i] -
 preferences [i] + 2
 for i in range (len (Schedule))

num_particles = 30
 num_iterations = 50

swarm = [Particle (len (preferences), bounds) for
 _ in range (num_particles)]

global_best = None

global_best_score = float ('inf')

for iteration in range (num_iterations):
 for particle in swarm:

score = fitness (particle.position)

~~particle~~

if score < particle.best_score:

particle.best_score = score

particle.best_position = ~~particle~~

~~particle~~ list (particle.position)

if score < global_best_score:

global_best_score = score

global_best = list (
 particle.position)

Code:

```
import random

# Objective function: power consumption =  $x^2 + y^2$ 
def power_consumption(position):
    x, y = position
    return x**2 + y**2

# PSO parameters
num_particles = 30
num_iterations = 100
w = 0.5      # inertia weight
c1 = 1.5     # cognitive coefficient
c2 = 1.5     # social coefficient

# Search space bounds
x_min, x_max = 0.1, 2.0  # thickness in mm
y_min, y_max = 1.0, 10.0 # length in cm

# Initialize particles
particles = []
for _ in range(num_particles):
    position = [random.uniform(x_min, x_max), random.uniform(y_min, y_max)]
```

```

velocity = [random.uniform(-1, 1), random.uniform(-1, 1)]
particles.append({
    'position': position,
    'velocity': velocity,
    'best_position': position[:],
    'best_score': power_consumption(position)
})

# Find global best
global_best = min(particles, key=lambda p: p['best_score'])
global_best_position = global_best['best_position'][:]
global_best_score = global_best['best_score']

# PSO loop
for iteration in range(num_iterations):
    for particle in particles:
        # Update velocity
        for i in range(2): # x and y
            r1, r2 = random.random(), random.random()
            cognitive = c1 * r1 * (particle['best_position'][i] - particle['position'][i])
            social = c2 * r2 * (global_best_position[i] - particle['position'][i])
            particle['velocity'][i] = w * particle['velocity'][i] + cognitive + social

        # Update position
        for i in range(2):
            particle['position'][i] += particle['velocity'][i]
            # Clamp position to bounds
            if i == 0:
                particle['position'][i] = max(x_min, min(x_max, particle['position'][i]))
            else:
                particle['position'][i] = max(y_min, min(y_max, particle['position'][i]))

        # Evaluate fitness
        score = power_consumption(particle['position'])

        # Update personal best
        if score < particle['best_score']:
            particle['best_position'] = particle['position'][:]
            particle['best_score'] = score

        # Update global best
        if score < global_best_score:
            global_best_position = particle['position'][:]
            global_best_score = score

# Output result
print("Best design found:")
print(f" Filament thickness (x): {global_best_position[0]:.4f} mm")
print(f" Filament length (y): {global_best_position[1]:.4f} cm")

```

PROGRAM 3

Ant Colony Optimization for the Traveling Salesman Problem

Algorithm:

Ant Colony Optimization

initialize pheromone values: $\tau_{ij} \rightarrow \tau_0$
 for each ant $k \in \{1, \dots, m\}$

initialize solution set $S = \{1, \dots, n\}$
 randomly choose starting city $i_0 \in S$
 move to starting city i_0
 while $S \neq \emptyset$

remove current city from 'S'

choose next city j with $p_{ij} =$

$$\frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{h \in S} \tau_{ih}^\alpha \cdot \eta_{ih}^\beta}$$

solution vector $\pi_k(i) \rightarrow j$

move to next city $i \rightarrow j$

finalize solution vector $\pi_k(i) \rightarrow j_0$

for each solution $\pi_k, k \in \{1, \dots, m\}$

calculate tour length $f(\pi_k) \rightarrow \sum_{i=1}^n d_{j_i, j_{i+1}}$

for all (i, j)

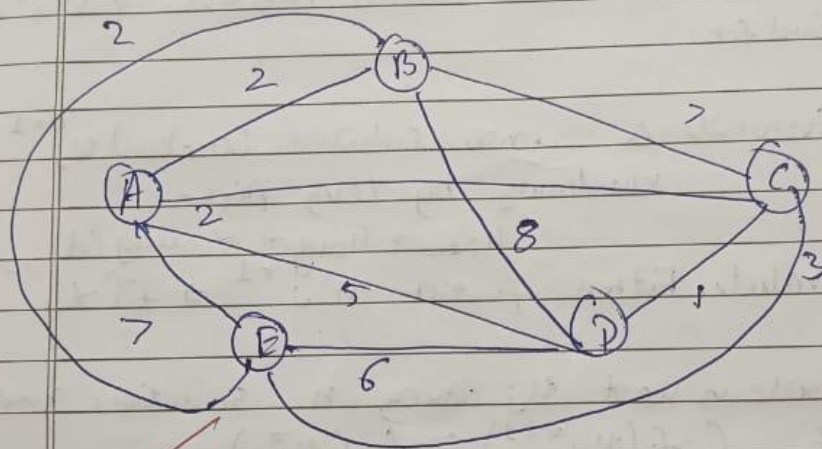
update pheromone $\tau_{ij} \rightarrow$
 $(1-p) \cdot \tau_{ij}$

determine best solution of iteration $\pi^+ =$
 $\arg \min_k f(\pi_k)$
 $k \in [1, m]$

if π^+ better than current best π^+
 $\pi^* \mapsto \pi^+$

for all $(i, j) \in \pi^+$
 reinforce $\tau_{ij} \mapsto \tau_{ij} + \Delta/2$

for all $(i, j) \in \pi^*$
 reinforce $\tau_{ij} \mapsto \tau_{ij} + \Delta/2$



$A \rightarrow B \rightarrow E \rightarrow C \rightarrow D \rightarrow A$ (cost = 13)
 2 2 3 1 5

Iteration 1: Best path length = 13

2: Best path length = 13

3: Best path length = 13

4: Best path length = 13

5: Best path length = 13

9/10

Code:

```
import numpy as np

# Distance between cities
def distance(city1, city2):
    return np.linalg.norm(np.array(city1) - np.array(city2))

# Initialize pheromone levels
def initialize_pheromones(num_cities, initial_pheromone):
    return np.full((num_cities, num_cities), initial_pheromone)

# Choose next city based on pheromone and heuristic info
def choose_next_city(current_city, unvisited, pheromone, distances, alpha, beta):
    pheromone_vals = pheromone[current_city, unvisited] ** alpha
    heuristic_vals = (1 / distances[current_city, unvisited]) ** beta
    probs = pheromone_vals * heuristic_vals
    probs /= probs.sum()
```

```

return np.random.choice(unvisited, p=probs)

# Compute total length of a tour
def tour_length(tour, distances):
    length = 0
    for i in range(len(tour) - 1):
        length += distances[tour[i], tour[i+1]]
    length += distances[tour[-1], tour[0]] # return to start
    return length

# Main ACO function
def ant_colony_optimization(cities, num_ants=10, num_iterations=100, alpha=1, beta=5,
evaporation=0.5, Q=100):
    num_cities = len(cities)
    distances = np.zeros((num_cities, num_cities))
    for i in range(num_cities):
        for j in range(num_cities):
            distances[i][j] = distance(cities[i], cities[j])

    pheromone = initialize_pheromones(num_cities, initial_pheromone=1.0)

    best_tour = None
    best_length = float('inf')

    for iteration in range(num_iterations):
        all_tours = []
        all_lengths = []

        for ant in range(num_ants):
            tour = []
            unvisited = list(range(num_cities))
            current_city = np.random.choice(unvisited)
            tour.append(current_city)
            unvisited.remove(current_city)

            while unvisited:
                next_city = choose_next_city(current_city, unvisited, pheromone, distances, alpha, beta)
                tour.append(next_city)
                unvisited.remove(next_city)
                current_city = next_city

            length = tour_length(tour, distances)
            all_tours.append(tour)
            all_lengths.append(length)

        if length < best_length:
            best_length = length
            best_tour = tour

```

```

# Evaporate pheromone
pheromone *= (1 - evaporation)

# Deposit pheromone proportional to quality
for tour, length in zip(all_tours, all_lengths):
    deposit_amount = Q / length
    for i in range(num_cities - 1):
        a, b = tour[i], tour[i+1]
        pheromone[a][b] += deposit_amount
        pheromone[b][a] += deposit_amount
    # Add pheromone for return edge
    a, b = tour[-1], tour[0]
    pheromone[a][b] += deposit_amount
    pheromone[b][a] += deposit_amount

if iteration % 10 == 0 or iteration == num_iterations - 1:
    print(f'Iteration {iteration+1}, best length: {best_length:.2f}')

return best_tour, best_length

# Example usage:

cities = [
    (0, 0), (1, 5), (5, 2), (6, 6), (8, 3)
]

best_tour, best_length = ant_colony_optimization(cities)

print("Best tour:", best_tour)
print("Best length:", best_length)

```

PROGRAM 4:

Cuckoo Search (CS)

ALGORITHM:

Cuckoo Search ~~Algorithm~~

Set initial value of host size n
 probability P_a
 Maximum iteration - max

Set $t = 0$

For ($j = 1, j \leq n$) do

Generate initial population of n host u_j^0

Evaluate Fitness function $f(x_j^0)$

End for

Generate a new solution (Cuckoo) u_j^{t+1}
 randomly by Levy flight

$$u_{new} = u_{old} + \alpha \cdot \text{Levy}(d)$$

Evaluate Fitness function $u_j^{t+1} \Rightarrow f(x_j^{t+1})$

Choose a nest u_j among n solutions randomly

If ($f(u_j^{t+1}) > f(u_j^t)$)

Replace solution u_j with u_j^{t+1}

End If

Abandon a fraction P_a of worst nest

Build new nest at new location using

Levy flight a fraction P_a of worst nest

Keep the best solution (nest with quality sol)

Rank the solution and find best

Set $t = t + 1$

Until ($t > max$)

Produce best solution

Initialize $\rightarrow f(x) = x^2 - 1$

Nests $\rightarrow u_1 = 4$

$u_2 = -3$

$u_3 = 6$

Max = 5

$P_a = 0.25$

Old best fitness = 8

New solution $\Rightarrow u_1 = 2$

$u_2 = -2$

$u_3 = 2$

New best fitness = 3

Updated

Abandon Random nest with P_a : $u_3 = \cancel{6} \rightarrow 2$

New best fitness = 0

Iteration 1 best fitness = 3 at $u = 2$

Iteration 2 best fitness = 3 at $u = 2$

Iteration 3 best fitness = 3 at $u = 2$

Iteration 4 best fitness = 0 at $u = 1$

Iteration 5 best fitness = 0 at $u = 1$

Optimal solution at $u = 1$

16/10/23

CODE:

```
import numpy as np
import random

# Sigmoid activation function and its derivative for
neural network
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Define a simple feedforward neural network
def neural_network(weights, inputs):
```

```

input_layer = inputs

hidden_layer = sigmoid(np.dot(input_layer,
weights['W1']) + weights['b1'])
output_layer = sigmoid(np.dot(hidden_layer,
weights['W2']) + weights['b2'])
return output_layer, hidden_layer

# Fitness function (Mean Squared Error)
def fitness_function(weights, inputs, outputs, target):
    predictions, _ = neural_network(weights, inputs)
    error = np.mean((predictions - target) ** 2)
    return error

# Initialize nests (cuckoos) with random weights
def initialize_nests(population_size, input_size,
hidden_size, output_size):
    nests = []
    for _ in range(population_size):
        nest = {
            'W1': np.random.randn(input_size, hidden_size),
            'b1': np.random.randn(hidden_size),
            'W2': np.random.randn(hidden_size,
output_size),
            'b2': np.random.randn(output_size)
        }
        nests.append(nest)
    return nests

# Lévy flight for generating new solutions
def levy_flight(current_nest, levy_step_size):
    new_nest = {
        'W1': current_nest['W1'] + levy_step_size *
np.random.randn(*current_nest['W1'].shape),
        'b1': current_nest['b1'] + levy_step_size *
np.random.randn(*current_nest['b1'].shape),
        'W2': current_nest['W2'] + levy_step_size *
np.random.randn(*current_nest['W2'].shape),
        'b2': current_nest['b2'] + levy_step_size *
np.random.randn(*current_nest['b2'].shape)
    }
    return new_nest

# Replace a fraction of nests with random solutions
def replace_nests_with_random_discovery(nests,
discovery_rate, input_size, hidden_size, output_size):
    num_replace = int(discovery_rate * len(nests))

```

```
for i in range(num_replace):
    nests[i] = {
        'W1': np.random.randn(input_size, hidden_size),
        'b1': np.random.randn(hidden_size),
```

```

        'W2': np.random.randn(hidden_size,
output_size),
        'b2': np.random.randn(output_size)
    }
    return nests

# Cuckoo Search main function
def cuckoo_search(inputs, target, population_size,
max_iterations, discovery_rate, levy_step_size):
    input_size = inputs.shape[1] # Number of input
features
    hidden_size = 5 # Hidden layer size (can be adjusted)
    output_size = target.shape[1] # Number of output
neurons (1 for regression or number of classes for
classification)

    # Initialize nests (cuckoos)
    nests = initialize_nests(population_size, input_size,
hidden_size, output_size)

    # Track the best nest
    best_nest = nests[0]
    best_fitness = fitness_function(best_nest, inputs,
target, target)

    # Main loop of Cuckoo Search
    for iteration in range(max_iterations):
        for i in range(population_size):
            # Evaluate fitness of the current nest (cuckoo)
            fitness = fitness_function(nests[i], inputs, target,
target)

            # If the fitness is better, update the best solution
            if fitness < best_fitness:
                best_nest = nests[i]
                best_fitness = fitness

            # Generate new candidate solution via Lévy
flight
            new_nest = levy_flight(nests[i], levy_step_size)

            # Evaluate fitness of the new solution
            new_fitness = fitness_function(new_nest, inputs,
target, target)

            # If the new solution is better, replace the old one
            if new_fitness < fitness:
                nests[i] = new_nest

    # Replace some nests with random solutions
(discovery rate)

```

```

        nests =
replace_nests_with_random_discovery(nests,
discovery_rate, input_size, hidden_size, output_size)

        # Print the progress
        print(f'Iteration {iteration + 1}/{max_iterations},
Best Fitness: {best_fitness}')

    return best_nest

# Main function to run the neural network training with
Cuckoo Search
if __name__ == "__main__":
    # User-defined parameters
    population_size = int(input("Enter population size: "))
    max_iterations = int(input("Enter max iterations: "))
    discovery_rate = float(input("Enter discovery rate
(between 0 and 1): "))
    levy_step_size = float(input("Enter Levy step size: "))

    # Example data (XOR problem, you can replace with
your data)
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    target = np.array([[0], [1], [1], [0]]) # XOR outputs

    # Train the neural network using Cuckoo Search
    best_weights = cuckoo_search(inputs, target,
population_size, max_iterations, discovery_rate,
levy_step_size)

    # Final trained model's weights
    print("\nFinal Trained Weights:")
    print("W1:", best_weights['W1'])
    print("b1:", best_weights['b1'])
    print("W2:", best_weights['W2'])
    print("b2:", best_weights['b2'])

    # Optionally test the model
    predictions, _ = neural_network(best_weights, inputs)
    print("\nPredictions on the training set:")
    print(predictions)

```

PROGRAM 5:

Grey Wolf Optimizer (GWO)

Algorithm:

Crazy Wolf Optimization

Set initial population of wolves $\rightarrow$

alpha-population, beta-population,
~~gamma~~^{delta}-population, omega-population

Evaluate-Fitness (wolf)

if fitness > 0.75

Set as alpha

elif fitness > 0.5 || fitness < 0.25

Set as beta

elif fitness > 0.15 || fitness < 0.5

Set as delta

else

Set as omega

Distance to Prey (X_p)

random no. $\rightarrow r$; $C \rightarrow r * 2$

$$D = \text{abs}(C * X_p - X)$$

Wolf position updates (n, d)

$r1 \leftarrow \text{random}$

$$A \leftarrow 2 * d * r1 - d$$

For loop (until convergence):

Evaluate-Fitness (wolves)

~~calculate~~
~~update~~

Distance to Prey (C)

Wolf-position-update (C)

return alpha-wolf

O/P →

For electrical load dispatch application
 - optimize to meet demand and if not met, apply penalty

Total power demand = 100

Iteration 20: Best cost (Alpha) = \$1695.2

Iteration 40: Best cost (Alpha) = \$1572.3

Iteration 60: Best cost (Alpha) = \$1529.6

Iteration 80: Best cost (Alpha) = \$1524.9

Iteration 100: Best cost (Alpha) = \$1529.6

6/1/23

CODE:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# -----
# Fitness Function
# -----
def fitness_function(features, X_train, X_test, y_train, y_test):
    # If no features selected → very bad fitness
    if np.sum(features) == 0:
        return 1e9
```

```

# Select features
X_train_sel = X_train[:, features == 1]
X_test_sel = X_test[:, features == 1]

# Train & evaluate classifier
clf = DecisionTreeClassifier()
clf.fit(X_train_sel, y_train)
y_pred = clf.predict(X_test_sel)
acc = accuracy_score(y_test, y_pred)

# Fitness: lower is better
return (1 - acc) + (np.sum(features) / len(features))

# -----
# Grey Wolf Optimization
# -----
def GWO(num_wolves, max_iter, num_features, X_train, X_test, y_train, y_test):
    # Initialize wolves randomly (binary vectors)
    wolves = np.random.randint(0, 2, (num_wolves, num_features))

    # Evaluate fitness
    fitness = np.array([fitness_function(w, X_train, X_test, y_train, y_test) for w in wolves])

    # Identify alpha, beta, delta
    sorted_idx = np.argsort(fitness)
    alpha, beta, delta = wolves[sorted_idx[:3]]

    for t in range(max_iter):
        a = 2 - 2 * (t / max_iter) # Decreasing from 2 to 0

        for i in range(num_wolves):
            if np.array_equal(wolves[i], alpha) or np.array_equal(wolves[i], beta) or
np.array_equal(wolves[i], delta):
                continue

            # Position update based on alpha, beta, delta
            for j in range(num_features):
                r1, r2 = np.random.rand(), np.random.rand()
                A1, C1 = 2*a*r1 - a, 2*r2
                D_alpha = abs(C1 * alpha[j] - wolves[i][j])
                X1 = alpha[j] - A1 * D_alpha

                r1, r2 = np.random.rand(), np.random.rand()
                A2, C2 = 2*a*r1 - a, 2*r2
                D_beta = abs(C2 * beta[j] - wolves[i][j])
                X2 = beta[j] - A2 * D_beta

                r1, r2 = np.random.rand(), np.random.rand()

```

```

A3, C3 = 2*a*r1 - a, 2*r2
D_delta = abs(C3 * delta[j] - wolves[i][j])
X3 = delta[j] - A3 * D_delta

wolves[i][j] = 1 if ((X1 + X2 + X3) / 3) > 0.5 else 0

# Re-evaluate fitness
fitness = np.array([fitness_function(w, X_train, X_test, y_train, y_test) for w in wolves])
sorted_idx = np.argsort(fitness)
alpha, beta, delta = wolves[sorted_idx[:3]]

return alpha

# -----
# Run Example
# -----
if __name__ == "__main__":
    # Load dataset
    data = load_breast_cancer()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.3, random_state=42
    )

    # Run GWO
    best_features = GWO(num_wolves=10, max_iter=20, num_features=X_train.shape[1],
        X_train=X_train, X_test=X_test, y_train=y_train, y_test=y_test)

    print("Selected features:", np.where(best_features == 1)[0])

```

PROGRAM 6:

Parallel Cellular Algorithms and Programs

ALGORITHM:

Parallel Cellular Algorithm

Define function to minimize: $f(x)$

Initialize Parameters - Grid size ($h \times h$)
No. of cells (h^2)
Iterations (it)

Initialize Population - Randomly assign values to cells between ~~0 and 1~~ -1 and 1

Evaluate fitness -

Evaluate fitness of each cell based on $f(x)$

Update status -

For each cell:

Calculate average fitness of neighbours of the cell

if ~~cell fitness~~ $'cell fitness' > 'neighbour fitness'$

$cell fitness \leftarrow neighbour fitness$

Main:

Initialize function, parameter, population

While 'convergence' or $it - current < it$:

~~For~~ For cells:

Evaluate Fitness

Update status

$min = \infty$

For cells:

$min = min(min, cell-fitness)$

C/P → ~~For~~ Online Equation solver

Minimize $f(x) = x^2 - 4x + 4$

Initialize parameters: $h = 5$

$it = 100$

$(-r, r) = (-10, 10)$

Min value after iteration 20: 9

Min value after iteration 40: 1

Min value after iteration 60: 0

Min value after iteration 80: 0

Min value after iteration 100: 0

Final min value: 0

Sol
20/1/23

CODE:

```
import numpy as np
```

```
# Parameters
```

```
L = 30          # Road length
```

```
Vmax = 5
```

```
p_slow = 0.3
```

```
timesteps = 10
```

```
num_cars = 5
```

```
# Initialize road: -1 for empty, else speed of car
```

```
road = -1 * np.ones(L, dtype=int)
```

```
car_positions = np.random.choice(L, num_cars, replace=False)
```

```
road[car_positions] = 0
```

```
def update_road(road):
```

```
    new_road = -1 * np.ones_like(road)
```

```
    L = len(road)
```

```

for i in range(L):
    if road[i] != -1: # There's a car here
        v = road[i]

        # Calculate gap to next car
        distance = 1
        while distance <= Vmax:
            check_pos = (i + distance) % L
            if road[check_pos] != -1:
                break
            distance += 1
        gap = distance - 1

        # Step 1: Acceleration
        v = min(v + 1, Vmax)

        # Step 2: Slowing down due to cars
        v = min(v, gap)

        # Step 3: Random slow down
        if v > 0 and np.random.random() < p_slow:
            v -= 1

        # Step 4: Move car
        new_pos = (i + v) % L

        # Check for collisions (should not happen if rules are correct)
        if new_road[new_pos] == -1:
            new_road[new_pos] = v
        else:
            # If collision, keep old position (very unlikely)
            new_road[i] = v

return new_road

def print_road(road):
    print("".join(['1' if x != -1 else '0' for x in road]))

print("Initial state:")
print_road(road)

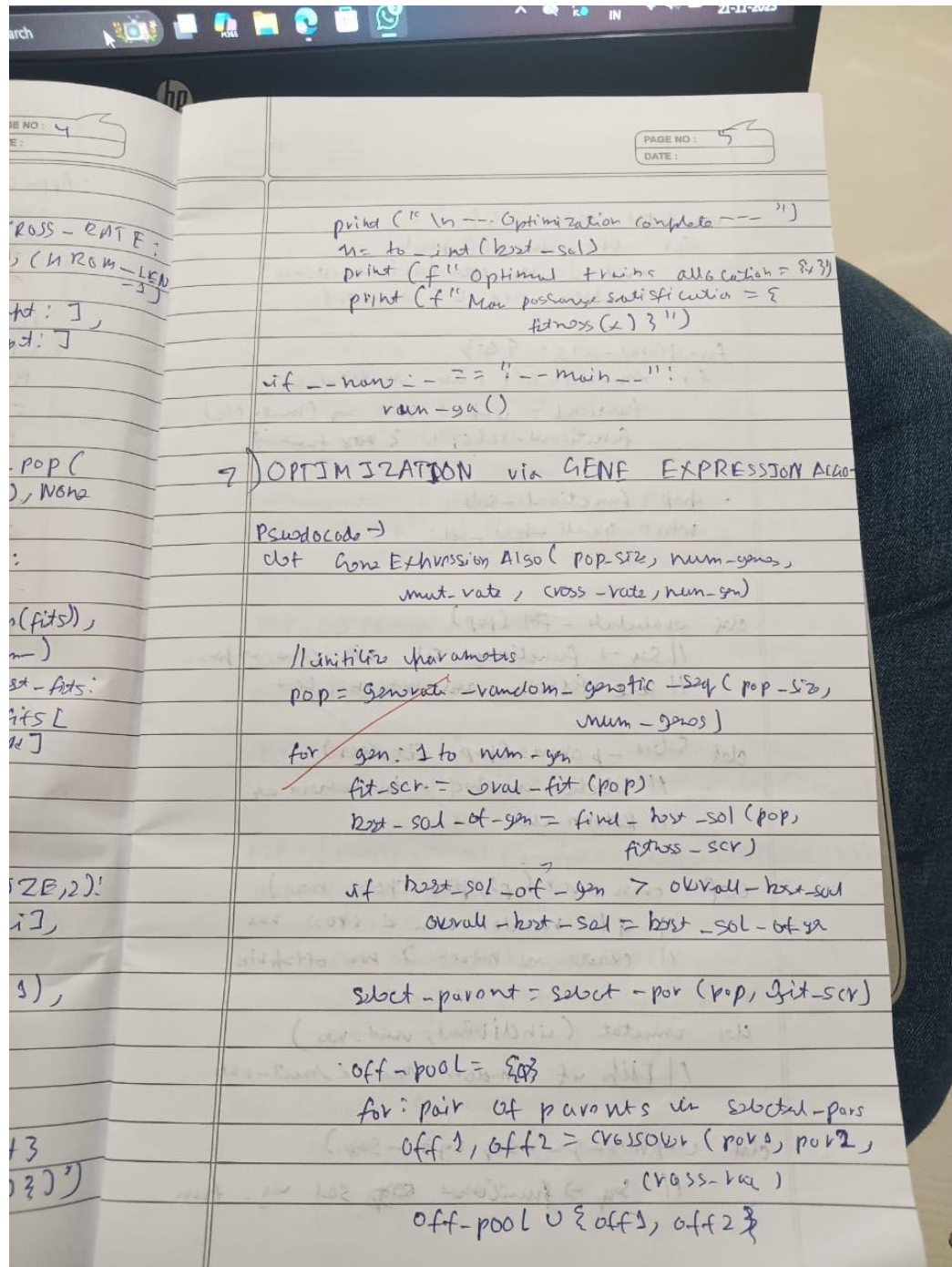
for t in range(timesteps):
    road = update_road(road)
    print(f'Step {t + 1} :')
    print_road(road)

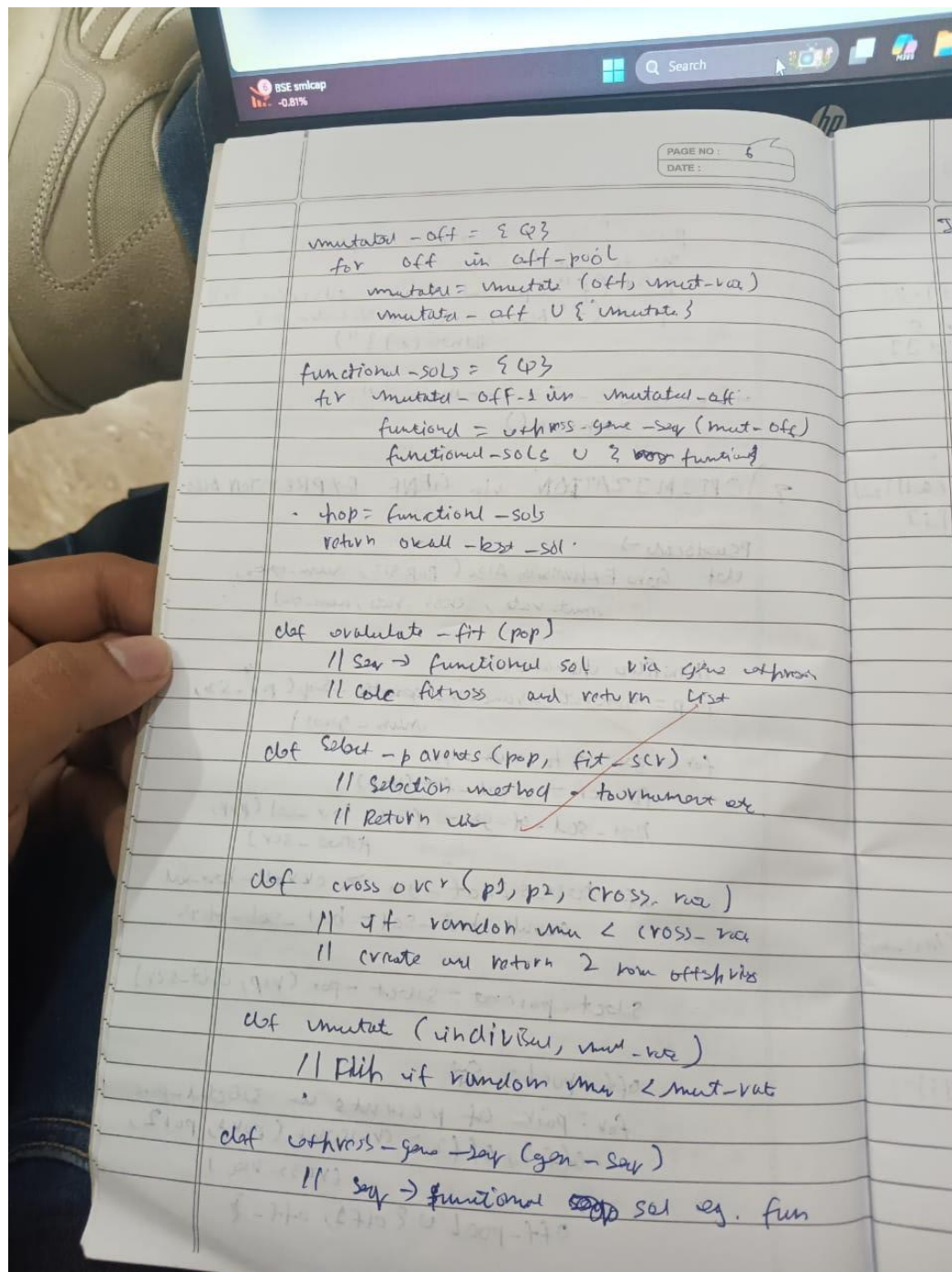
```

PROGRAM 7:

Optimization via Gene Expression Algorithms

ALGORITHM:





CODE:

```

import random
import math
  
```

Cities (x, y coordinates)

```

cities = [(0,0), (1,3), (4,3), (6,1), (3,0)]
  
```

```
# Distance between two cities
```

```
def dist(a, b):
```

```
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
```

```
# Total route distance
```

```
def route_distance(route):
```

```
    return sum(dist(cities[route[i]], cities[route[(i+1)%len(route)]]) for i in range(len(route)))
```

```
# Fitness (shorter distance = better)
```

```
def fitness(route):
```

```
    return 1 / route_distance(route)
```

```

# Create initial population
def init_population(size):
    base = list(range(len(cities)))
    return [random.sample(base, len(base)) for _ in range(size)]

# Selection (roulette wheel)
def select(pop, fits):
    total = sum(fits)
    pick = random.uniform(0, total)
    curr = 0
    for i, f in enumerate(fits):
        curr += f
        if curr > pick:
            return pop[i]

# Crossover (ordered crossover)
def crossover(p1, p2):
    a, b = sorted(random.sample(range(len(p1)), 2))
    child = [None]*len(p1)
    child[a:b] = p1[a:b]
    ptr = 0
    for x in p2:
        if x not in child:
            while child[ptr] is not None:
                ptr += 1
            child[ptr] = x
    return child

# Mutation (swap two cities)
def mutate(route):
    i, j = random.sample(range(len(route)), 2)
    route[i], route[j] = route[j], route[i]
    return route

# Main GA loop
POP_SIZE = 4
GENERATIONS = 5
pop = init_population(POP_SIZE)

for gen in range(GENERATIONS):
    fits = [fitness(r) for r in pop]
    best_route = pop[fits.index(max(fits))]
    print(f'Gen {gen+1}: Best distance = {route_distance(best_route):.2f}, Route = {best_route}')

    new_pop = []
    for _ in range(POP_SIZE):
        p1, p2 = select(pop, fits), select(pop, fits)
        child = crossover(p1, p2)
        if random.random() < 0.2:

```

```
        child = mutate(child)
    new_pop.append(child)
    pop = new_pop
```

```
# Final Result
```

```
fits = [fitness(r) for r in pop]
best_route = pop[fits.index(max(fits))]
print("\nFinal Best Route:", best_route)
print("Final Best Distance:", route_distance(best_route))
```